

Комплексно решение на упражнението: EchoDiary

Име и идея на приложението

Името на приложението е **EchoDiary**. Това е personal music diary уеб приложение, което помага на потребителя да разглежда Spotify слушанията си не само като статистика, а като личен дневник. Основната идея е музиката да бъде представена като част от ежедневието, настроенията и различните периоди от живота на потребителя.

Приложението не цели просто да покаже „най-слушани песни“ или „най-слушани изпълнители“. То трябва да даде контекст: кои песни са слушани в конкретни дни, кои от тях са liked, колко често потребителят слуша любими песни и кои изпълнители доминират в последната му listening history.

Основни потребителски възможности

Потребителят трябва да може да:

- влиза със Spotify акаунт чрез OAuth 2.0 Authorization Code with PKCE;
- вижда Spotify profile информацията си;
- разглежда recent listening activity в хронологичен дневников изглед;
- вижда кои песни от историята са liked/saved;
- вижда статистика за процента liked songs;
- разглежда интерактивни графики по изпълнител;
- сменя metric в графиките между track count и listening time;
- вижда детайли за всяка песен: име, изпълнител, албум, cover, duration, popularity и timestamp.

Предложена архитектура

EchoDiary трябва да бъде server-backed web application. Frontend частта отговаря само за визуализацията и interaction layer, а backend частта управлява Spotify authentication, token exchange, token refresh, sessions и Spotify API calls.

Browser Client

|
| HTTPS + HttpOnly Secure Cookie

v

Web Backend

| -- Auth Controller
| -- Spotify API Service
| -- Session / Token Store
| -- Aggregation Service

Този подход е избран, защото access tokens и refresh tokens не трябва да се съхраняват в browser local storage. Клиентът получава само secure session cookie, а реалните Spotify tokens остават на сървъра.

Authentication flow

Приложението използва **OAuth 2.0 Authorization Code with PKCE**. Spotify описва PKCE flow като подходящ за приложения, при които client secret не може безопасно да се пази в client-side среда, а flow-ът включва code verifier, code challenge, authorization code exchange и API calls със session token ([Spotify Authorization Code with PKCE Flow](#)).

Login процес

1. Потребителят натиска **Sign in with Spotify**.
2. Backend-ът генерира code_verifier, code_challenge и state.
3. Потребителят се redirect-ва към Spotify /authorize.
4. В authorization request се подават client_id, response_type=code, redirect_uri, scope, state, code_challenge_method=S256 и code_challenge.
5. Добавя се show_dialog=true, за да се показва consent/account choice екран всеки път; Spotify описва show_dialog като параметър, който принуждава потребителя отново да одобри приложението вместо автоматичен redirect ([Spotify Implicit Grant Flow](#)).
6. Spotify връща callback към /auth/callback.
7. Backend-ът валидира state, обработва евентуални грешки и обменя code за tokens чрез /api/token.
8. Tokens се записват encrypted в server-side session store.
9. Потребителят се redirect-ва към /profile.

Token management

Access token и refresh token се пазят само server-side. При изтичане на access token backend-ът извършва automatic refresh. Browser-ът не получава Spotify token и не може да го прочете чрез JavaScript.

Spotify scopes

Минималните scopes за първата версия са:

- user-read-private за базова Spotify profile информация;
- user-read-email за email, защото Spotify посочва, че email field е наличен при granted user-read-email scope ([Spotify Get Current User's Profile](#));
- user-read-recently-played за recent listening activity;
- user-library-read за проверка дали песните са saved/liked.

Първата версия е read-only. Ако по-късно се добави save/unsave функционалност, ще трябва да се добавят write scopes.

Основни страници

Home page

Home page представя EchoDiary с кратък текст: „Your Spotify listening history as a personal music diary“. В header-а има **Sign in with Spotify** бутон, който се вижда само когато потребителят не е authenticated.

Profile page

След успешен login потребителят винаги се redirect-ва към Profile page. Тя използва Spotify GET /me, който връща текущия user profile, включително display_name, email и images ([Spotify Get Current User's Profile](#)).

Profile page показва:

- display name;
- email, ако е наличен;
- profile picture;
- кратък call-to-action към Listening History.

Listening History page

Listening History page използва Spotify endpoint-а „Get Recently Played Tracks“. Този endpoint връща recent tracks, като всеки item съдържа track, played_at и context, а track object включва данни като name, artists, album, duration_ms, popularity и id ([Spotify Get Recently Played Tracks](#)).

Песните се показват хронологично и се групират по ден. Всеки card показва:

- име на песента;
- изпълнител или изпълнители;
- име на албума;
- album cover;
- duration;
- popularity score;
- timestamp кога е слушана;
- liked indicator.

Endpoint-ът за recently played има limit с максимум 50 items и поддържа after/before cursors, които не трябва да се използват едновременно ([Spotify Get Recently Played Tracks](#)).

Liked songs

За liked indicator се използва „Check User’s Saved Tracks“. Endpoint-ът проверява дали една или повече песни са saved в потребителската Spotify library, приема comma-separated ids параметър и връща array от boolean стойности, като максимумът е 50 IDs ([Spotify Check User’s Saved Tracks](#)).

Процесът е:

1. Вземат се recent tracks.
2. Track IDs се deduplicate-ват.
3. Изпраща се batch request към saved tracks endpoint.
4. Boolean резултатите се map-ват обратно към track cards.
5. Изчислява се процент liked songs.

Формулата е:

[liked_percentage =]

Insights page

Insights page показва статистики и графики:

- total recent tracks;
- liked songs percentage;
- total listening time;
- top artists;
- chart toggle между **Track count** и **Listening time**.

За artist chart може да се използва horizontal bar chart или donut chart. При много изпълнители се показват top 10, а останалите се групират като „Other“, за да остане UI четим.

Backend API design

Route	Method	Purpose
/auth/login	GET	Стартира Spotify login flow
/auth/callback	GET	Обработва Spotify callback
/auth/logout	POST	Изтрива session
/api/profile	GET	Връща Spotify profile data
/api/listening-history	GET	Връща recent tracks, групирани по ден
/api/stats/liked	GET	Връща liked percentage
/api/charts/artists?metric=track_count	GET	Връща artist chart по брой песни
/api/charts/artists?metric=listening_time	GET	Връща artist chart по слушано време

Data model

Session model

Field	Purpose
session_id	Internal session identifier
spotify_user_id	Spotify user ID
access_token_encrypted	Encrypted access token
refresh_token_encrypted	Encrypted refresh token
expires_at	Token expiry timestamp
scope	Granted scopes

Track view model

Field	Purpose
track_id	Spotify track ID
track_name	Song title
artists	Artist names
album_name	Album title
album_cover_url	Cover image
duration_ms	Track duration
popularity	Spotify popularity score
played_at	Timestamp
is_liked	Saved/liked state

Security requirements

Security е ключова част от решението:

- tokens не се пазят в local storage;
- session cookie е HttpOnly, Secure и с подходящ SameSite;
- state се използва срещу CSRF, тъй като Spotify го препоръчва като защита срещу cross-site request forgery ([Spotify Authorization Code with PKCE Flow](#));
- redirect_uri трябва да съвпада точно с регистрирания URI, защото Spotify изисква exact match ([Spotify Authorization Code with PKCE Flow](#));
- tokens, authorization codes и лични данни не се записват в logs;
- backend routes проверяват authenticated session преди достъп до Spotify data.

Performance и reliability

Приложението трябва да избягва излишни API calls. Recent tracks се взимат веднъж, saved tracks се проверяват batch-wise, а artist aggregations се изчисляват от вече получените данни.

UI трябва да показва:

- loading state при profile и listening history;
- skeleton cards при зареждане;
- empty state, ако няма recent listening data;
- error state при Spotify API грешка;
- retry option при временен проблем.

Charts трябва да се render-ват smooth и да не се обновяват ненужно при всяка малка промяна в UI.

Accessibility и UX

Liked songs не трябва да се показват само с цвят. Трябва да има icon и text label, например „Liked“. Charts трябва да имат кратко текстово summary или fallback table. Всички controls, включително metric toggle, трябва да работят с keyboard navigation.

Дизайнът трябва да е responsive:

- mobile: single-column cards;
- tablet: grouped list with compact cards;
- desktop: history list + insights panel.

AI workflow с GitHub Copilot Agents

Тъй като упражнението изрично казва, че приложението няма да се имплементира, GitHub Copilot Agents трябва да се използват за планиране, архитектура, validation и automation design, а не за писане на production code.

Предложени agents

Agent	Role	Output
Product Agent	Превръща requirements в user stories	Product brief и acceptance criteria
Architecture Agent	Проектира secure architecture	Architecture Decision Record
Security Agent	Проверява OAuth, tokens и privacy risks	Security checklist
UX Agent	Описва screens и states	UX flow map
Data Agent	Дефинира transformations и aggregations	Data spec

Agent	Role	Output
QA Agent	Подготвя test strategy	Test matrix

Примерен workflow

Exercise requirements

- > Product Agent: user stories and scope
- > Architecture Agent: system design
- > Security Agent: OAuth and token review
- > UX Agent: page flows and states
- > Data Agent: liked songs and chart logic
- > QA Agent: validation matrix
- > Human review and final plan

Примерни prompts

Product Agent

You are the Product Agent for EchoDiary.

Turn the requirements into user stories, acceptance criteria, and out-of-scope items.

Focus on the product experience and the diary metaphor.

Do not implement code.

Architecture Agent

Design a secure web architecture for a Spotify OAuth 2.0 Authorization Code with PKCE application.

The browser must never store access tokens.

Include auth routes, session storage, token refresh, Spotify API integration, and error handling.

Return the result as an Architecture Decision Record.

Security Agent

Review the EchoDiary architecture for OAuth, PKCE, CSRF, token storage, redirect URI handling, logging, and session cookie security.

Return risks, severity, recommendations, and validation methods.

Data Agent

Define the data transformations for recent Spotify tracks, liked-song mapping, liked percentage, artist track count, and artist listening time.

Include edge cases such as duplicate tracks, missing images, local tracks, empty history, and max 50 recent items.

QA Agent

Create a test plan for EchoDiary.

Cover OAuth success and failure, token refresh, protected routes, profile rendering, recent tracks grouping, liked indicator mapping, chart toggling, loading states, and accessibility.

Acceptance criteria

Authentication

- Sign in button се вижда само когато потребителят не е logged in.
- Spotify consent/account choice се показва при всяко login действие.
- Callback route обработва success, denied access и unexpected errors.
- Tokens не се пазят в local storage.
- Token refresh работи автоматично.

Profile

- След login потребителят се redirect-ва към profile page.
- Profile page показва name, email и profile image, когато са налични.
- При липсваща profile image има fallback.

Listening history

- Recent tracks се групират по ден.
- Всеки track card показва name, artists, album, cover, duration, popularity и played timestamp.
- Liked songs имат visual indicator и text label.
- Empty history показва разбираемо съобщение.

Charts and stats

- Liked percentage се изчислява правилно.
- Artist chart поддържа track count и listening time.
- Metric toggle сменя данните без full page reload.
- Top artists са четими и при дълги имена.

Non-functional

- Има loading indicators.
- API calls са optimized и batch-нати.
- Sensitive data не се логва.
- Layout-ът е responsive.
- Интерфейсът е keyboard accessible.

Рискове и решения

Risk	Impact	Mitigation
Spotify recent history е ограничена	Потребителят може да очаква пълна история	Ясно label-ване като recent activity
Token leakage	Security incident	Server-side token storage only
Missing profile data	Broken UI	Fallback avatar и conditional fields
Chart clutter	Трудна четимост	Top 10 + Other grouping

Risk	Impact	Mitigation
API errors	Лош UX	Loading, retry и error states
Multiple artists per track	Aggregation ambiguity	Ясно правило за aggregation

Финално заключение

EchoDiary е добре планирано personal music diary приложение, което комбинира Spotify authentication, profile data, recent listening history, liked-song analysis и artist-based insights. Най-важното архитектурно решение е всички чувствителни tokens да се пазят server-side, а browser-ът да работи само със secure session cookie.

GitHub Copilot Agents трябва да се използват не за директна имплементация, а за структуриран planning workflow: product definition, architecture, security review, UX states, data transformations и QA strategy. Така упражнението изпълнява целта си: да подобри уменията за планиране, архитектурно мислене и ефективна работа с AI agents.

Източници

- Spotify Authorization Code with PKCE Flow: <https://developer.spotify.com/documentation/web-api/tutorials/code-pkce-flow>
- Spotify Implicit Grant Flow: <https://developer.spotify.com/documentation/web-api/tutorials/implicit-flow>
- Spotify Get Current User's Profile: <https://developer.spotify.com/documentation/web-api/reference/get-current-users-profile>
- Spotify Get Recently Played Tracks: <https://developer.spotify.com/documentation/web-api/reference/get-recently-played>
- Spotify Check User's Saved Tracks: <https://developer.spotify.com/documentation/web-api/reference/check-users-saved-tracks>